

강화학습

강화학습이란?

강화학습 개념 등장

"강화학습" 개념은, 1979년 A. Harry Klopff의 "heterostatic theory of adaptive systems"에서 나온 "hedonistic learning system" 이라는 개념에서 유래되었다. 이를 번역하면 "쾌락주의 학습 방법"으로, 강화학습의 핵심 아이디어인 "무언가를 원하는 방식의 학습 시스템"과 큰 맥락이 비슷하다고 볼 수 있다.

강화학습 초기

강화학습 분야 발전에서의 초기 단계에는 Computational studies of learning에 초점이 맞춰져 있었다. 즉, 문제를 해결하기 위한 하나의 알고리즘 연구로 진행되었다. 그에 따라 "환경으로부터 원하는 것을 얻는 방법"을 학습하는 것에 대한 연구가 적은 주목을 받았다.

하지만 "학습 개념"에 초점을 맞춘 근본적인 아이디어 자체를 탐구해야 오히려 Computational study의 진보가 가능하기 때문에, 강화학습 자체에 대한 근본적인 연구가 선행되어야 한다.

강화학습의 중요 역할

강화학습은 optimal control과 dynamic programming 이론들 사이의 관계를 정립시키고 발전시켰다.

optimal control : 주어진 목표를 달성하기 위해 어떤 policy를 따라야 하는지에 대한 연구

dynamic programming : 최적문제를 풀기 위한 수학적 도구(optimal의 방법론 중 하나)로 하나의 문제는 단 한번만 풀게하는 알고리즘

강화학습 개념에 대한 선행 연구 결과

temporal difference learning, dynamic programming, function approximation에 대해 일관된 관점 내에서 정리 가능해졌다. 이 책은 강화학습의 핵심 개념과 알고리즘을 설명한다.

Sutton 책의 구성

크게 4부분으로 구성되어 있다.

1. Intro

강화학습 정의와 특징

2. Small finite state space에서의 표 기반 방법 소개

- action value을 추정하는 기본적인 방법 소개
- DP, MC, TD에 대해 소개
- Eligibility Traces를 통해 MC와 TD를 설명
- Planning method(DP, state-space search), learning method(MC, TD)에 대해 설명한다.

3. 표 기반 방법을 확장한 방법들

4. 강화학습의 생물학적 기원과 응용사례

Sutton 책 학습 방법

1장으로 강화학습 공부 → 2장에서 2.2장까지 공부 → 3장에서 4,5,9절을 제외하고 공부

⇒ 관심사에 따라 나머지 공부

단, 2장은 연계성이 중요하여 차례대로 학습하고 6절이 가장 중요하다.

9절 : 머신러닝, 신경망 관점에서 공부할 때 중요

8절 : AI, Planning 관점에서 중요하다.

Chapter 1 : The Reinforcement Learning Problem

강화학습의 추상적인 관점 :

“우리가 환경과 상호작용하며 학습한다”

이 개념은 학습의 본질에 대해 생각할 때 가장 먼저 드는 생각이다.

예를 들면, 아이가 행동을 할 때, 명확하게 가르쳐준 교사가 없지만 아이는 환경과 직접적인 감각-운동 연결을 맺고 있다. 이 “연결”을 활용하면서 아이는 인과관계, 행동의 결과, 목표를 이루기 위한 방법 등에 대한 풍부한 정보를 얻는다.

이 장에서는 Interaction을 통한 학습을 computational 접근으로 살펴본다. 이상적인 학습 상황을 설정하고, 다양한 학습방법의 효과를 평가한다.

목표 : 과학적이나 경제적으로 중요한 학습문제를 효과적으로 해결할 수 있는machine을 설계한다.

이 때, 강화학습 접근으로 진행할 것 이고, 이는 goal - directed learning from intersection에 초점을 둔 방법이다.

Reinforcement Learning

<강화학습이란?>

하나의 문제이자, 이 문제를 효과적으로 해결할 수 있는 방법들의 집합이면서, 이러한 문제와 해결방법들을 연구하는 분야

<강화학습 문제>

수치적 보상 신호를 최대화하기 위하여,

1. 무엇을 학습할 것 인가?
2. 어떻게 상황을 행동에 맵핑할 것 인가?

이러한 전략을 세팅하고 나면, 한 학습 사이클이 끝나고 그 결과가 다음 사이클에 영향을 미치기 때문에, 본질적으로 Closed-loop problem이다.

<강화학습특징 + 머신러닝과의 차이>

강화학습 : 어떤 행동을 취해야 하는지 알려주지 않는다. 대신, 어떤 행동이 가장 많은 보상을 가져오는지 직접해보면서 찾아낸다.

다만, 대부분의 머신러닝은 어떤 행동을 어떻게 취해야하는지 명시한다.

머신러닝은 크게 3가지로 나눈다면, Supervised / Unsupervised / Reinforcement learning

<Supervised learning>

사전지식을 가지고 있는 외부 교사가 제공하는 정답을 포함한 데이터로 학습함

1. 데이터에는 특정 상황에 대한 설명이 있음
2. 데이터에는 해당 상황에서 취해야 할 올바른 행동을 명시하는 테이블을 포함

⇒ 상황이 어떤 Category에 속하는지 판별한다.

목적 : 학습된 데이터가 아니더라도 새로운 상황에서 올바른 행동을 수행할 수 있도록 응답 일반화

<Unsupervised learning>

라벨이 없는 데이터에서 자료의 구조를 찾는 것이 목표인 학습방법

<Reinforcement learning>

정확한 행동의 예시를 기반으로 하지 않는다. 하지만 강화학습의 본질은 숨겨진 구조를 찾는 것이 아니라, 보상신호를 최대화 하는 것 이다. 숨겨진 구조를 찾는 것이 유용할 수는 있지만, 그 자체로 보상을 최대화 하는 문제를 해결하지는 않는다.

⇒ Machine Learning의 3번째 타입이라고 할 수 있다.

<강화학습의 중요한 3가지 요소>

1. 본질적으로, Closed-loop 형태 일 것.
2. 어떤 행동을 취할 것인지에 대해 직접적인 지시가 없을 것
3. 보상을 포함한 행동결과가 오랜 시간에 걸쳐 결과로 나타나는 환경

<강화학습에서의 에이전트>

에이전트 필수 조건 세 가지

1. 환경의 상태를 어느정도 파악할 수 있어야 한다. (Sensation)
2. 환경의 상태에 영향을 미치는 행동을 할 수 있어야 한다. (action)
3. 환경의 상태와 관련된 목표가 있어야 한다. (goal)

세 요소가 포함되는 구조가 강화학습 Agent의 필수 요건이고, 이런 세팅에서 문제를 풀 수 있는 방법들을 강화학습이라고 통칭한다.

미지의 환경(기존에 경험하거나 학습한 적이 없는 새로운 환경이나 상황)에서는 에이전트가 자신의 경험으로부터 학습할 수 있어야 한다.

<Supervised learning과의 차이점>

강화학습에서는 에이전트가 행동해야 하는 모든 상황을 포괄하면서 그 때마다 올바른 행동을 주는 예제들을 얻는 것이 종종 비현실적이다. 즉, 상호작용을 통한 학습의 매우 큰 데이터를 학습시키는 것이 불가능하다고 볼 수 있다.

<강화학습의 고유한 문제>

Exploration과 Exploitation간의 trade-off가 다른 학습방법들(나머지 두개 머신러닝)에서는 크게 고려되지 않는 요소이지만, 강화학습에서는 매우 중요하고 어려운 문제이다.

에이전트는 이미 알고 있는 최선의 선택을 exploit하여 보상을 얻어야 하지만, 더 나은 행동을 찾기 위해 explore도 해야 한다.

⇒ 어느 하나만 독단적으로 사용하면 과제에서 실패하게 되는 딜레마가 있다.

따라서, 에이전트는 다양한 행동을 시도하면서 점진적으로 가장 좋아보이는 것을 선택해야 한다.

<강화학습 특징-에이전트, 환경>

불확실한 환경에서 상호작용하는 목표지향적인 agent에 대한 전체적인 문제를 명시적으로 고려해야 한다. subproblem을 설계하여 해결할 때도 관점을 확장을 어떻게 하여 real problem에 적절히 맞출지, 가능할지 고려해야 한다.

~~처음에 강화학습 연구자들은 supervised learning 연구에 집중했지만, 이 학습방법이 어떻게 유용하게 사용될 것인지 논의가 안되었다. 일반적인 목표를 가진 planning 이론을 개발했지만, 이러한 계획이 실시간 의사결정에서 planning이 어떠한 역할을 하는지, planning에 필요한 예측 모델이 어디에서부터 비롯되는지에 대한 의문이 제기되지 않았다. 즉, 개별적인 subproblem에 초점을 두는 것은 한계점이 있다...~~

<강화학습 접근 방식>

Complete, interactive, goal-seeking 한 Agent를 가지고 시작한다.

** 하위문제들에 대해 연구할 때도 그것들이 Complete, interactive, goal-seeking면에서 명확한 역할이 있어야 한다.

Agent :

1. 명확한 목표를 가지고 있다.
2. 환경의 양상(상태)를 파악 가능하다.
3. 환경에 영향을 주는 행동을 선택할 수 있다.

가정 : 에이전트가 마주한 상당히 불확실한 환경을 감안하고 작동하는 것으로 시작한다.

<강화학습이 Planning을 포함하는 경우>

Planning과 실시간 행동선택 관계를 고려해야 한다. 또한 환경모델에서 보상이 어떻게 획득되고 개선되는지에 대한 문제를 해결해야 한다.

Examples

- Master chess player가 수를 둘 때 응수(예상되는 수)를 고려해야하고 특정한 위치에 수를 둘 때 즉각적이고 직관적인 판단을 한다.
- 석유정제소에서 Adaptive controller는 순이익을 기반으로 처음에 엔지니어가 설정해놓은 고정 값이 아니라 유동적으로 yield, cost, quality를 최적화 한다.
- 가젤은 태어날 때 걷지도 못하지만, 30분이 지나면 시속 20마일로 뛰어다닌다.
- 움직이는 로봇이 새로운 방에 들어가서 쓰레기를 찾아 버리고 배터리를 충전한다. 이 각각의 의사결정 시스템은 배터리가 얼마 없을 때 충전을 할지 마저 일을 하고 충전할지 등이 있을 수 있고 이 의사결정은 이전에 충전을 진행할 때의 데이터, 쓰레기를 버린 동선에 대한 데이터들을 기반으로 진행한다.
- 아침밥을 준비하는 과정에서 조건부 행동들의 연결들과 goal-subgoal간의 연계성들을 고려해야한다. 찬장까지 걷고, 이것을 열고, 시리얼 박스를 선택하고, 손을 뻗고, 박스를 들어 가져오는 이러한 행동 절차들이 있다. 각 step의 행동 자체에도 개별 목표가 있고, 이 개별목표들이 모여서 더 큰 목표가 된다.

<예제로부터 알아보는 강화학습>

공통적으로 interaction을 갖고 있다.

- 능동적으로 의사결정하는 Agent와 환경이 있다. (Agent는 환경의 불확실성에서도 goal 달성을 목표로 한다.)
- Agent의 행동이 다음 시간대에 Agent가 가질 수 있는 선택지나 기회에 영향을 미친다.

특징 1 : 행동의 결과를 정확히 예측할 수 없다.

특징 2 : 설정된 goal은 명시적이다. 또한 Agent는 인지, 감각을 통해 직접적으로 목표 달성 진행현황을 알 수 있다.

특징 3 : Agent는 경험을 통해 시간이 지남에 따라 성능 개선이 가능하다.

<Agent와 Environment>

Environment에는 Agent의 상태, 기억, 목표가 포함될 수 있다. Agent는 이러한 경험들을 가지고 시간이 지남에 따라 수행능력을 증진시킨다. 환경과의 상호작용은 특정 과제의 특성을 활용할 수 있게 행동을 조정하는데 필수적이다.

<Policy, Reward signal, value function>

Policy : Agent가 특정 시점에서 행동하는 방식을 결정하는 요소로, 감지된 환경 상태에서 어떻게 행동할지를 맵핑하는 함수

⇒ Policy 만으로도 행동 결정 가능하고, 일반적으로 Stochastic이다.

Reward signal : 강화학습 문제에서의 목표로, time step 마다 Environment이 Agent에게 single number 로 reward를 주는 것 이다. 이는 action과 state에 따라 변동된다. 일반적으로 policy를 변경하는 기본적인 기준이 된다.

****Agent가 reward에 영향을 미칠 수 있는 유일한 방법은 action이다.**

⇒ 직접적인 영향 - 보상 수치

간접적인 영향 - Environment state 변경

Value function : 장기적인 관점에서 무엇이 좋은지 나타낸다. 특정 state의 value function은 그 state로 시작했을 때 얻을 수 있는 보상의 총량을 나타낸다.

**** Reward signal과 Value function의 차이**

Reward signal : 환경 상태의 즉각적인 유용성 결정

Value function : 이 상태 이후의 상태들, 그 상태들로부터의 보상을 고려한다.

따라서, 현재의 reward 가 낮더라도 잠재력이 크면 value 는 높아진다

이에 반해, 지금 당장의 reward가 높더라도 잠재력이 작으면 value는 작아진다.

Agent가 경험을 바탕으로 value를 추정하고 지속적인 평가를 진행하고, 최종 학습 목표 중요도는 reward보단 value에 초점을 둔다. 그래도 value의 목적 자체는 reward를 최대화 하는 것 이다.

Model of the environment : 환경의 동작을 모방하거나 일반적으로 환경이 어떻게 동작할지를 추론하는 것이다. 주어진 state와 action에 대해 해당 모델은 결과로 나타낼 next state와 next reward를 예측할 수 있다. 모델은 planning을 위해 사용되는데, 이는 실제로 경험하기 전에 가능한 미래 상황을 고려하여 행동 방침을 결정하는 방식을 의미한다.

미래의 상황을 실제로 경험하기 전 고려하여 행동 방침을 결정하는 방식이 model-based method

이에 반해 trial-and-error를 학습을 수행하는 방법들을 model-free method (planning과 정 반대)

<가치함수가 필수적인가?>

이 책에서의 강화학습 방법은 Value function을 추정하는 구조를 기반으로 하지만, 강화학습 문제를 해결하기 위해 반드시 Value function을 추정하는 구조를 기반으로 하지만, 강화학습문제를 해결하기 위해 반드시 vlaue function을 설계해야하는 것만은 아니다.

EX) GA, GP SA

<Evolutionary Methods>

개별적인 Agent 가 직접 학습하지 않고 여러개의 non learning Agent를 생성 후, 그 중 가장 성능이 좋은 에이전트를 선택하여 다음 세대로 전달.

Multi-arm Bandits (Tabular Solution Method)

강화학습의 가장 큰 구별되는 특징은 Training information가 올바른 행동을 지시하는 것이 아니라, 수행된 행동을 평가한다는 점이다. 이러한 특성 때문에 active exploration이 필요하며, 좋은 행동을 찾기 위한 trial-and-error이 필수적이다.

Purely evaluative feedback은 수행된 행동이 얼마나 좋은지를 나타내지만, 그것이 가능한 행동 중 최선인지 최악인지는 나타나지 않는다. 이러한 평가적 피드백은 Evolutionary method를 포함한 함수 최적화 기법들의 기초가 된다.

반면, purely instructive feedback은 실제로 수행된 행동과 관계없이, 올바른 행동이 무엇인지를 직접적으로 지시한다. 이것이 supervised learning의 기초가 된다.

Purely evaluative feedback : 수행된 행동에 의존하여 그에 따른 reward 제공 (사전 정보x)

purely instructive feedback : 행동과 무관하게 정답을 알려준다.

이번 장 목표 : 단일 상황에서 학습하는 문제를 다루며, 평가적 피드백과 지도적 피드백의 차이점과 이를 어떻게 결합하는지에 대해 이해

An n-Armed Bandit Problem

n개의 서로 다른 행동 중 하나를 반복적으로 선택

각 선택 후 해당 선택에 따라 결정되는 확률분포에서 선택된 numerical reward 받음

목표 : 일정 기간동안 기대 총 보상을 최대화

이 문제에서 각 action은 선택될 때 기대되는 평균 보상을 가진다. 이 기대 보상을 해당 행동의 가치 라고 할 수 있다. 만약 모든 행동의 가치를 정확히 알고 있다면, 문제를 해결하는 것은 간단하다. 다만 이는 현실에서 그렇지 않아, 이 때 Exploitation 와 Exploration 개념이 나오게 되는 것.

Exploitation은 greedy action 선택.

Exploration은 일부러 greedy action을 선택하지 않음.

탐색을 하면 단기적으로는 손해를 볼 수 있지만, 장기적으로 더 큰 보상을 얻을 수 있다. 하지만 탐색과 활용을 동시에 수행할 수는 없으므로, 이 둘 사이의 균형을 맞추는 것이 중요한 문제.

일반적으로,

- 시간이 충분히 남아 있다면 탐색을 수행하여 더 나은 행동을 찾는 것이 유리함
- 시간이 얼마 남지 않았다면 활용을 통해 현재까지 학습한 최적 행동을 선택하는 것이 유리함

이 두가지를 적절히 조합하는 다양한 수학적 방법들이 존재한다.

Action-Value Methods

행동의 가치를 추정하는 간단한 방법과 이 추정값을 이용하여 행동을 선택하는 방법을 살펴보자.

$q(a)$: 행동 a 의 실제 가치

$Q_t(a)$: t 번째 시간 단계에서의 추정된 가치

행동의 실제 가치는 그 행동이 선택될 때 평균적으로 받을 수 있는 보상을 의미

이를 추정하는 것은 그 행동이 선택될 때 실제로 받은 보상의 평균을 계산하는 것.

t 번째까지 a 가 $N_t(a)$ 번 선택되고, 그동안 받은 보상이 $R_1, R_2, \dots, R_{N_t(a)}$

이 때, 가치 추정 : $Q_t(a) = \frac{R_1 + R_2 + \dots + R_{N_t(a)}}{N_t(a)}$

~~이외의 가치 추정 방법들이 많다.~~

<greedy 행동 선택 방법>

가장 단순한 행동 선택 방법으로, t 번째 시간단계에서 greedy action A_t 를 선택하는 것 이다.

이 방법은 현재까지 학습된 정보를 최대한 활용하여 즉각적인 보상을 극대화 한다. 그러나, 이 방법은 더 나은 행동이 있을 가능성이 있는 다른 행동을 탐색할 기회를 완전히 무시한다.

< ϵ -greedy 방법>

greedy 방법의 단점은 항상 현재의 정보만을 활용하기 때문에 장기적인 최적 행동을 찾을 가능성이 낮다. 이의 보완으로 ϵ -greedy 방법 제시.

동작 방식 : $1-\epsilon$ 확률로 greedy, ϵ 확률로 무작위 행동

즉, 확률 ϵ 로 모든 행동을 확률적으로 선택하며 이를 통해 아직 충분히 탐색되지 않은 행동을 시도할 수 있다.

장점 :

- 모든 행동이 무한 번 샘플링 되어, $N_t(a) \rightarrow \infty$ 가 보장된다.
- $Q_t(a)$ 는 $q(a)$ 에 수렴하게 된다.
- 이는 최적 행동을 선택할 확률이 $1-\epsilon$ 로 수렴

이는 무한히 많은 시행을 반복할 때 성립되는 것이다. 하지만 실제로는 충분히 많은 시행을 함으로써 greedy 방법의 단점을 일부 보완하는 형식으로 진행된다.

~~이와 같이, Exploitation과 Exploration의 적절한 조화를 위해 시도되는 방법들을 ϵ -greedy가 아니더라도 샘플링을 랜덤하게 진행하거나 일정 제약조건을 주어 일부러 탐색을 시도하는 방법을 적용해야 하는 것이 중요할 것이다.~~

~~스도쿠에서 적용한다고 하면, 열추 알고리즘이 돌아가면서 들어가는 숫자가 고착화 되고 기존에 들어갈 수 있었던 수들이 배척되는 상황을 방지하는 것이 될텐데, 이를 어떻게 탐색할지에 대해 체계적인 과정이 필요할 것이다.~~

Incremental Implementation

지금까지 다룬 action-value 방법은 모두 관찰된 보상의 샘플 평균을 사용하여 행동 가치를 추정했다.

가장 명확한 구현 방법은 각 행동 a 에 대해, 해당 행동을 선택한 이후 받은 모든 보상을 저장하는 것 이다.

이 때, 단순히 $Q_t(a) = \frac{R_1 + R_2 + \dots + R_{N_t(a)}}{N_t(a)}$ 를 계속 계산하는 것이 아니라, 이전에 사용했던 계산을 사용하여 점진적으로 업데이트함으로써 메모리와 연산량을 줄일 수 있다.

$$\begin{aligned}
 Q_{k+1} &= \frac{1}{k} \sum_{i=1}^k R_i \\
 &= \frac{1}{k} \left(R_k + \sum_{i=1}^{k-1} R_i \right) \\
 &= \frac{1}{k} (R_k + (k-1)Q_k + Q_k - Q_k) \\
 &= \frac{1}{k} (R_k + kQ_k - Q_k) \\
 &= Q_k + \frac{1}{k} (R_k - Q_k)
 \end{aligned}$$

Tracking a Nonstationary Problem

강화학습문제의 대부분은 본질적으로 정적인 상태가 아니다. 즉, 시간에 따라 환경이 바뀌고, 이에 따라 행동의 가치 (보상의 기대값)이 변할 수 있다.

이 때, LSTM 구조와 비슷하게 오래된 보상보다 최근의 보상에 더 많은 가중치를 부여하는 방법을 채택한다. 여기서 보통 사용되는 방법이 constant step-size parameter α 를 사용한다.

위에 나와있는 식을 변형하여 $Q_{k+1} = Q_k + \alpha(R_k - Q_k)$ 로 업데이트하는 것이다.

여기서 α 는 상수이며, $0 < \alpha \leq 1$ 이다. 이렇게 하면 Q_{k+1} 는 과거 보상과 초기 추정값 Q_1 의 가중 평균이 된다.

** 이 식을 전개하면

$$Q_{k+1} = \alpha R_k + (1 - \alpha)\alpha R_{k-1} + (1 - \alpha)^2 \alpha R_{k-2} + \dots + (1 - \alpha)^{k-1} \alpha R_1 + (1 - \alpha)^k Q_1$$

이고, 이 가중치들의 합이 1이 되며 오래된 보상일수록 $1 - \alpha$ 씩 곱해지기 때문에 줄어들고, 이 이유로 가중 평균이라고 한다.

스도쿠에서 방법을 설계할 때 가장 핵심적인 포인트가 될 것 같다.

오래된 가중치에 적은 보상을 줄 경우에, 처음을 포함한 다양한 경우의 수 (트리형식으로 뿔어져 가는 경우의 수 구조)의 초반 부분을 바꿔가며 탐색할 수 있다. 하지만 이것이 끝까지 진행될 때 과연 brute-force 알고리즘과 시간적으로 얼마나 차이가 날지도 중요하게 봐야할 것 같다. 또한, 한번 숫자가 바뀌고 나면 얼마나 빠르게 숫자가 (합리적으로) 채워지는지에 따라서 제약조건을 추가로 걸면 더 좋을 것 같다. 예를 들면, 3/4/5가 후보군 일때 3을 넣고 계속해서 보상의 총량이 업데이트 될 때, 몇번째까지 갔을 때 얼마나 보상이 빠르게 올랐는지를 확인하고, 자연스럽게 초기 단계인 3을 넣었을 때의 상황이 점점 작아져 4를 택하게 되었을 때 보상의 총량의 상승 속도를 비교하여 둘 중 하나로 시작하는 경우를 없애버리는 방법이 있을 것 같다.

이 처럼, 강화학습을 스도쿠에 직접적으로 접목시킬 때는 점수 자체에 요점을 두는 것 보다도 점수의 상승 폭 혹은 점수가 오르는 속도라는 개념(다른 새로운 개념이 들어와도 됨)을 도입하여 초기 단계의 트리구조를 몇개 죽이며 경우의 수를 빠르게 줄이는 방식을 설계하는 것이 현재로써는 최선이라고 생각한다.

Optimistic Initial Values

초기 행동 가치 $Q_1(a)$ 를 단순히 0이 아니라 더 높은 값으로 설정하는 방법이다.

처음에 낮은 보상을 주게 되면 에이전트가 현재 보상에 실망하여 다른 선택을 하려고 하기 때문에, 초반 성능은 낮더라도 value에 관점을 두고 학습진행이 되는 점에서 유용한 방법이다.

~~이는 우리가 설계 단계에서 크게 고려하지 않아도 되는 부분인 것 같다.~~

Upper-Confidence-Bound Action Selection

exploration이 필요한 이유는 action value의 추정값이 불확실하기 때문이다. 현재 가장 좋아 보이는 greedy action을 선택한다고 해서, 실제로 그 행동이 최적 행동일 것이라는 보장은 없다. ϵ -greedy 행동 선택 방법은 비탐욕적 행동(non-greedy actions)도 시도하게 강제하지만, 그 과정이 무작위적이기 때문에, 탐욕적 행동에 가까운 행동이나 불확실성이 높은 행동을 더 우선적으로 선택하는 메커니즘이 없다. 즉, 탐색을 좀 더 효율적으로 할 수 있는 방법이 필요하다. 이러한 방법 중 하나가 Upper Confidence Bound, UCB방식이다.

UCB 방식에서는 각 행동의 가치 추정값에 불확실성을 반영한 추가 보너스를 더하여 행동을 선택한다.

$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

- $Q_t(a)$: 행동 a 의 현재 가치 추정값
- $N_t(a)$: 행동 a 가 지금까지 선택된 횟수
- t : 현재까지의 총 행동 선택 횟수
- $\ln t$: 자연로그(natural logarithm) ($e \approx 2.71$ 을 t 에 대해 몇 번 제곱해야 하는지를 나타냄)
- $c > 0$: 탐색 정도를 조절하는 하이퍼파라미터

즉, 탐색 가치가 다음 두 가지 요소에 의해 결정된다.

1. 현재 추정값 $Q_t(a)$ 가 높을수록 선택될 가능성이 커짐
 - 즉, 현재까지의 학습 결과, 좋은 보상을 받은 행동이 우선적으로 선택됨.
2. 불확실성이 클수록(즉, $N_t(a)$ 가 작을수록) 더 많이 선택됨
 - $N_t(a)$ 가 작으면, $\sqrt{\frac{\ln t}{N_t(a)}}$ 가 커지므로 탐색 보상이 커짐.
 - 따라서, 아직 많이 시도되지 않은 행동들이 자연스럽게 탐색됨.

결과적으로, 처음에는 모든 행동을 충분히 시도하면서 탐색을 수행하고, 시간이 지날수록 가치가 높은 행동을 더 자주 선택하는 방식이 된다.

GRADIENT BANDITS

(공부중)

Associative Search

지금까지는 비연관 작업만을 다루었다. 즉 각각의 상황에 따라 다른 행동을 연결할 필요가 없는 문제를 고려했다. 그러나, 일반적인 강화 학습 문제에서는 여러 개의 다른 상황(situations)이 존재하며, 목표는 각 상황에서 최적 행동을 찾는 정책(policy)을 학습하는 것이다.

무작위의 슬롯이 주어지면서 슬롯을 돌릴때, 슬롯머신의 identity를 알려주는 clue를 받는 경우를 생각. 슬롯머신 디스플레이가 바뀌는 상황 가정. 빨간색이면 슬롯 A, 초록색이면 슬롯 B. 이렇게 각 슬롯머신에 대한 최적 행동을 투트랙으로 세팅하고 각 상황에서의 policy 설계 가능.

아게 아제 스도쿠에서도 숫자가 세팅되면 그와 연쇄작용처럼 풀어가 될 것이라 중요한 부분이라고 생각했는데 생각보다 책에서도 간단히 다루고, 아게 정확히 어떤 것인지 조금 더 알아봐야 할 것 같다.

Finite Markov Decision Process (공부중)

출처 : <https://m.blog.naver.com/ehdrndd/221782051845>

<https://hijigoo.github.io/reinforcement%20learning/2019/01/31/rl-finite-markov-decision-processes/>

<https://courses.cs.washington.edu/courses/cse599i/18wi/resources/lecture17/lecture17.pdf>

<https://devslem.github.io/reinforcement-learning/rl-fundamental/finite-markov-decision-processes/>

Markov Decision Processes (MDPs)는 연속적인 의사 결정을 형식화한 프레임워크이다.

MDPs와 Multi-armed bandit 환경의 가장 큰 차이점은, MDPs에서는 선택한 행동(action)이 환경의 상태(state)를 변경시키고, 이를 통해 미래 보상(future rewards)에 영향을 미친다는 점이다.

즉, MDPs는 상태(state)와 행동(action)이 연관된(associative) 설정이다.

MDP는 다음과 같은 네 가지 요소로 구성된다:

- S : 상태 집합 (Set of states)
- A : 행동 집합 (Set of actions)
- P : 상태 전이 확률 함수 (State-transition probability function)
- R : 보상 함수 (Reward function)

즉, MDPs는 (S,A,P,R)로 표현할 수 있다.

~~스도쿠를 푸는데, 룰을 명시 안하는 것도 방법인것같다... 명시하고 풀이시키는 것이 아니라 리워드 점수~~

~~에 의해 숫자를 넣는 행위가 우리가 정한 조건에 맞게 넣을 수 있게...? 너무 산으로 가는 것 인지 아이디어
가 될 지 모르겠다.~~

MDPs에서 learner이자 decision maker를 agent라고 하며,

agent가 상호작용하는 agent 외의 모든 요소를 environment라고 한다.

Decision making은 agent가 action을 선택하는 행위를 의미한다.

Discrete time steps $t = 0, 1, 2, \dots$ 가 있을 때,

각 time step t 에서 agent는 environment의 state S_t 에서 action A_t 를 선택한다.

그러면 새로운 state S_{t+1} 로 전이되며,

environment로부터 numerical reward R_{t+1} 을 획득한다.

MDP의 개념을 정리하면 다음과 같다:

- 현재 상태: S_t
- 행동 선택: A_t
- 상태 전이: $S_t \rightarrow S_{t+1}$
- 보상 획득: R_{t+1}

MDP의 과정은 다음과 같이 순환된다:

$(S_t, A_t, R_{t+1}, S_{t+1}), (S_{t+1}, A_{t+1}, R_{t+2}, S_{t+2}), \dots$

즉, agent는 상태 S_t 에서 행동 A_t 를 수행하고, 새로운 상태 S_{t+1} 로 이동하며 보상 R_{t+1} 을 받는 과정이 반복된다.

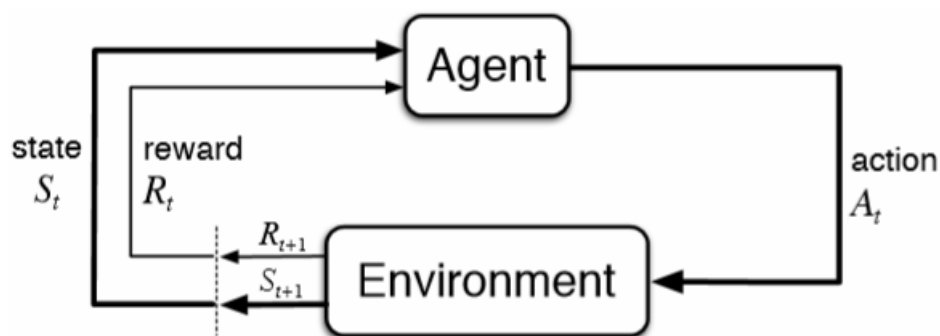


Figure 3.1: The agent–environment interaction in a Markov decision process.